

CS 35L Software Construction

Lecture 14 – Coding with GenAI

Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department

“I suspect that machines to be programmed in our native tongues—be it Dutch, English, American, French, German, or Swahili—are as damned difficult to make as they would be to use.” —Edsger W. Dijkstra (1979)



Learning Objectives of this Lecture

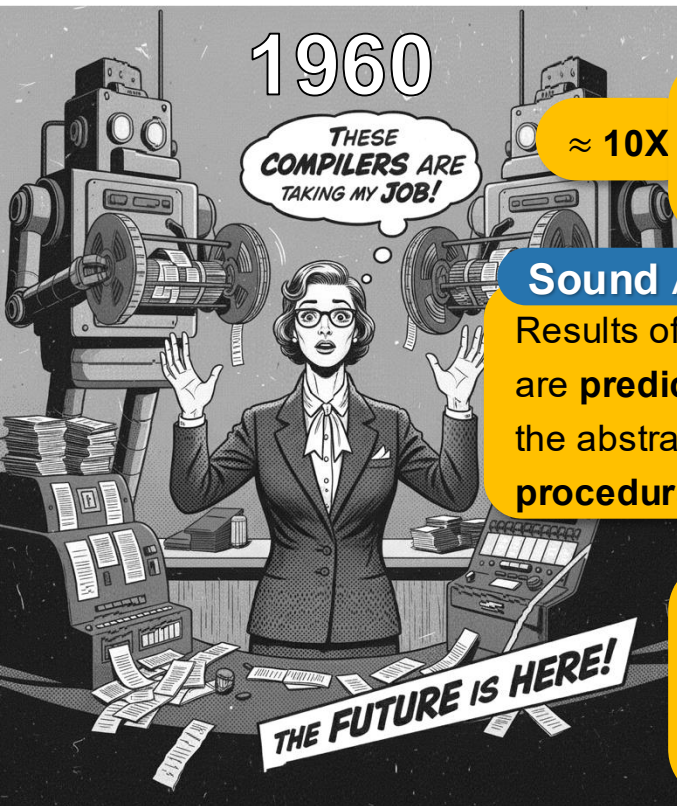
After this lecture you should be able to:

- Explain **how AI coding agents work**
- Identify **challenges & risk** that result from the use of AI coding agents
- Apply software engineering techniques to **mitigate the risks** of AI-generated code
- Apply **prompt engineering** techniques to **maximize the usefulness** of LLMs



What do these Scenarios have in Common?

What is Different now? Talk to your Neighbor(s)



1960

THESE
COMPILERS ARE
TAKING MY JOB!

≈ 10X

Productivity Gains

A single developer can write the code for much larger software systems

≈ 1.21X^[1]

– 1.5X^[2]

Sound Abstractions

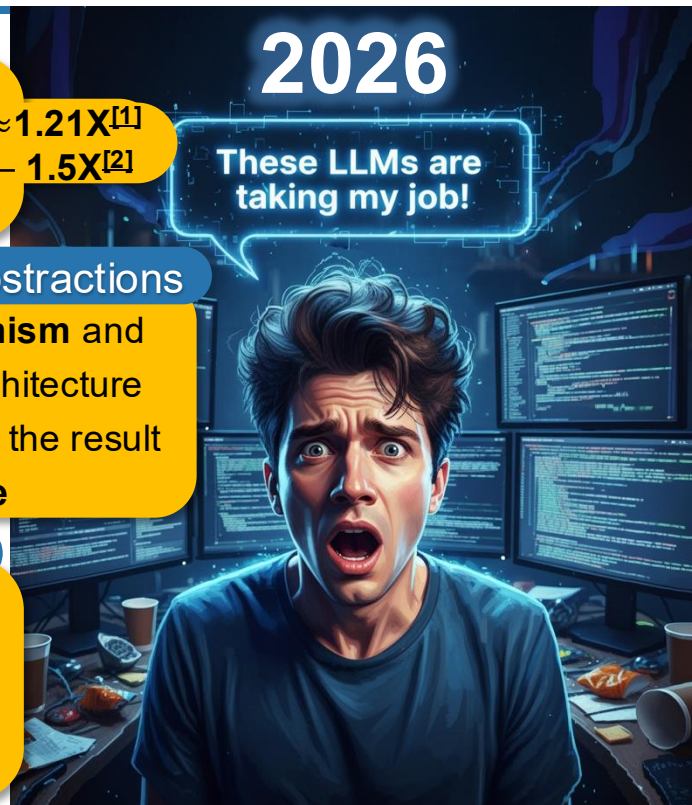
Results of a compiler are **predictable** since the abstractions are **procedural**

Unsound Abstractions

Non-determinism and **black-box** architecture of LLMs make the result **unpredictable**

Remaining Challenges

Requirements, design & quality assurance still requires **human intelligence** due to essential complexity



2026

These LLMs are
taking my job!



Thomas Dohmke

Former CEO of GitHub

@ashtom

Full Tweet: <https://x.com/ashtom/status/1952409910236291109>

“The evidence is clear: Either you embrace AI, or get out of this career.

Our latest field study with 22 developers who are integrating AI deeply into their workflows reveals a striking trend: those who persist beyond early skepticism emerge with dramatically higher ambition, technical fluency, and job satisfaction.

They’re not writing less code – they’re enabling more complex, system-level work through orchestration. [...] Here’s what we’re seeing:

- AI is on track to write 90% of code within the next 2–5 years.
- Developers aren’t worried. They’re optimistic and realistic about the changes ahead.
- **New skills matter now**, e.g. agent orchestration, iterative collaboration, and critical verification.
- **Time savings? Sure. But the real shift is ambition. Developers are raising the ceiling, not just lowering the cost.”**

See <https://ashtom.github.io/developers-reinvented>

Watch this interview <https://www.youtube.com/watch?v=3WNukz5-Ch0>

LLMs (Large Language Models) Work like “Statistical Parrots”

1. Pre-Training – Creating a base model / foundation model

The model is trained on vast amounts of code to accurately **predict the most likely next token given all previous tokens**.

Having **memorized** a large amount of code and **abstracted** key patterns in the code, the LLM can **reproduce variations** of the **most common coding tasks**.

2. Post-Training – Optimizing the model for a particular use case

- **Fine-tuning** the model on **labeled input-output pairs** (e.g., instructions + code solutions, such as LeetCode problems)
- Using **human feedback** to optimize the model using reinforcement learning (e.g., developers rank code outputs on correctness, efficiency, readability)

3. Inference – Prompting the model for a task

Based on a **natural language prompt**, the LLM tries to produce the most likely sequence of answer tokens (usually in a **non-deterministic way**)

```
def fib_iter(n):  
    a, b = 0, 1  
    for _ in range(n):  
        a, b = b, a + b  
    return a
```



code the **most efficient** and **shortest** Python implementation of **Fibonacci**

How Much Would You Trust A Parrot Witness?



Is likely to repeat what has been said many times before

Might repeat phrases **out of context** or from a **long time ago**

Might quote **factually-incorrect** statements

Might **make up** statements

Useful in some cases, but not 100% trustworthy

This is How Much You Should Trust a Parrot Coder (aka “AI Coding Agent”)

Useful
in many
cases, but
not 100%
trustworthy

Is likely to repeat
what has been coded
many times before

Might repeat **outdated**
code or code from
different context

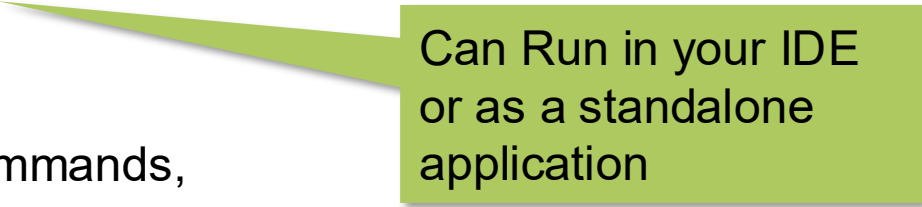
Might include **buggy**
or **insecure** code

Might **make up** calls
to non-existent code



Thinking / Reasoning Mode Takes More Time To answer more Intelligently

- **Thinking model** = the AI model spends extra inference compute on intermediate reasoning before answering
 - This “appears” like the model is thinking / reasoning
- **Coding agent** = an app/runtime around the model that gives it tools:
 - read files,
 - edit files,
 - run shell commands,
 - run tests,
 - inspect errors,
 - iterate.



Can Run in your IDE
or as a standalone
application

“Vibe Coding” – Are we there yet?

- [Andrej Karpathy](#): “There's a new kind of coding I call “vibe coding”, where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. [...]” [Feb 2, 2025](#)
- Vibe coder in [interview study](#): “Regulators don’t care about your “vibes”. Laws don’t care about your feelings” [1]
- Vibe coding might work for low-risk, internal or hobby projects. But it is not yet feasible for commercial enterprise projects
 - [Professional Software Developers Don't Vibe, They Control](#)
 - “I like coding alongside agents. Not vibe coding. But working ****with****.” [2]

[1] V. Pimenova, S. Fakhoury, C. Bird, C., M.A. Storey, M. Endres, “Good vibrations? A qualitative study of co-creation, communication, flow, and trust in vibe coding”. *arXiv preprint (2025)*

[2] R. Huang, A. Reyna, S. Lerner, H. Xia, B. Hempel, “Professional Software Developers Don't Vibe, They Control: AI Agent Use for Coding in 2025”. *arXiv preprint (2025)*

Generative AI Works Like an “**Amplifier**” Not a “Fixer”

- For **high-performing teams** with strong foundational practices (automated testing, robust CI/CD pipelines, strong platform engineering), **AI accelerates their success and throughput**
 - Clean architectures, good quality assurance processes increase productivity.
 - **More experienced developers receive a larger speedup from AI**
- For **struggling teams**, AI **magnifies their dysfunction**. Generating code faster without the right guardrails just means generating technical debt and bugs at an accelerated rate
 - Technical debt and implicit documentation makes it harder for AI to modify the code

Sources: 2025 DORA report by Google & E. Paradis, K. Grey, Q. Madison, D. Nam, A. Macvean, V. Meimand, N. Zhang, B. Ferrari-Church, S. Chandra. “How much does AI impact development speed? An enterprise-based randomized controlled trial.” *ICSE-SEIP '25*.

What Common Challenges Arise from the Use of Generative AI?



Use of AI Tools Resulted In Less Secure Code in Practice

- AI-generated code is **causing serious security vulnerabilities in 20% of companies** [1]
- Developers who had access to an AI assistant **wrote significantly less secure code** than those without access to an assistant while being **more likely to believe they wrote secure code** [2]
- Even when the LLM was explicitly instructed to *improve security*, it still introduced new vulnerabilities. The more complex the code, the more were vulnerabilities added by AI [3]
- **40% of code generated with GitHub Copilot contains security vulnerabilities** [4]

[1] State of AI in Security & Development.

[2] N. Perry et al. "Do Users Write More Insecure Code with AI Assistants?" CHI 2023

[3] S. Shukla et al. "Security Degradation in Iterative AI Code Generation: A Systematic Analysis of the Paradox" ISTAS 2025

[4] H. Pearce, et al. "Asleep at the keyboard? Assessing the security of Github Copilot's Code Contributions." Com ACM 2025

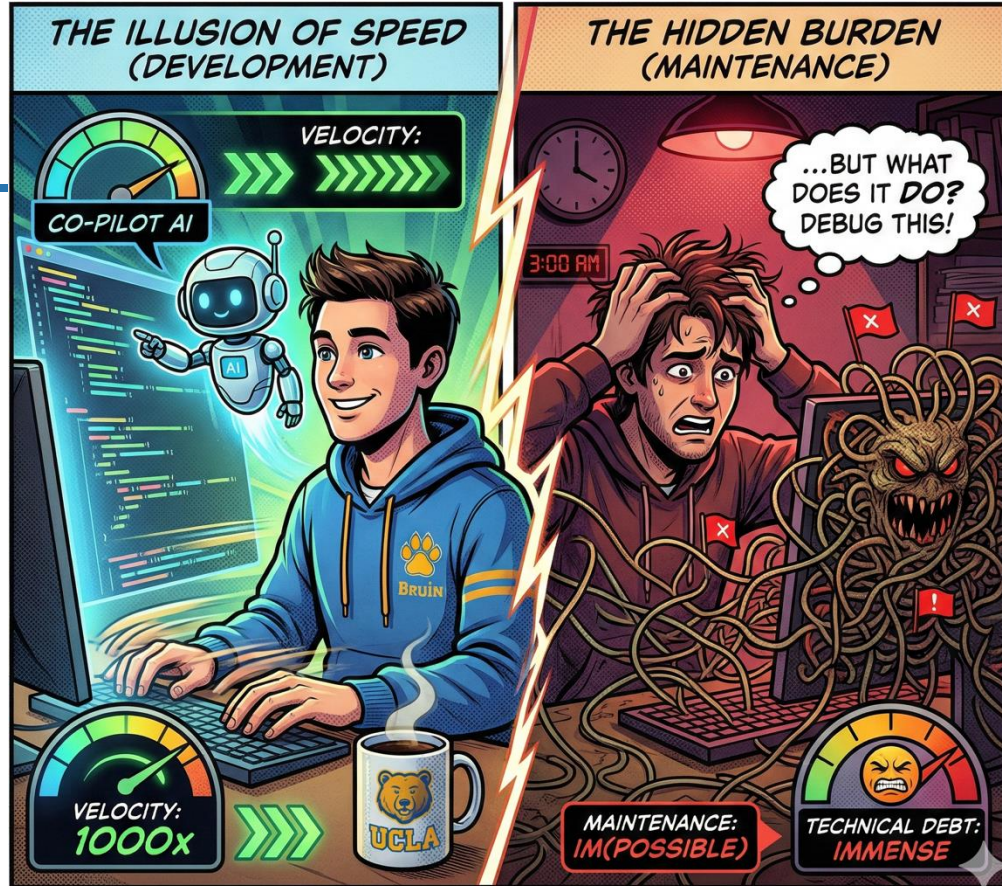
Code in the **training set is insecure**

I'm sorry for my mistake.
Here is some new (still incorrect) code for you

Think-Pair-Share: How do these results impact your approach to Software Engineering with GenAI?
Discuss actionable insights!

Speed at the Cost of Quality

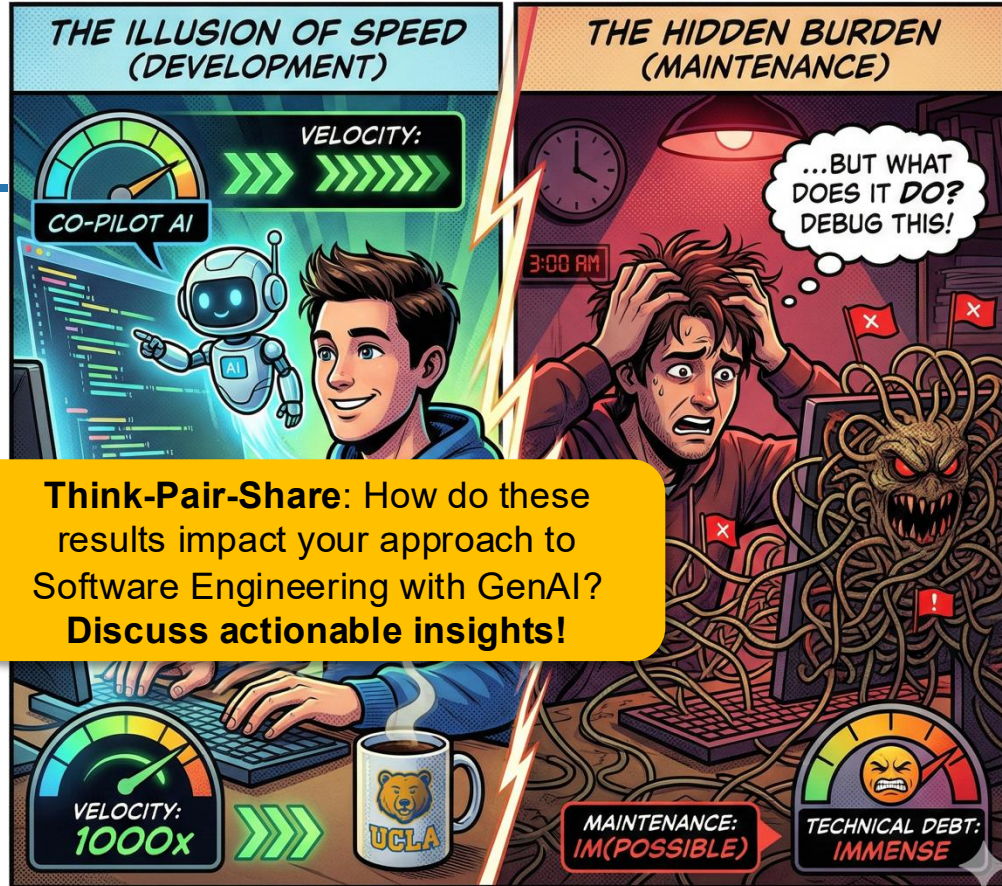
- How does the use of AI affect open-source projects?
- Study on 806 GitHub repos that used Cursor and 1380 similar non-Cursor repos
- Adopting Cursor resulted in a **massive but temporary surge in development velocity** (3x-5x increase in lines of code added during the first one to two months but disappeared shortly after)



Source: H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu. 2026. Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. MSR '26

Speed at the Cost of Quality

- How does the use of AI affect open-source projects?
- Study on 806 GitHub repos that used Cursor and 1380 similar non-Cursor repos
- 30.3% average **increase in static analysis warnings** and a 41.6% **increase in code complexity**
- technical debt generated by rapid AI usage actively **harms future productivity**



Think-Pair-Share: How do these results impact your approach to Software Engineering with GenAI?
Discuss actionable insights!

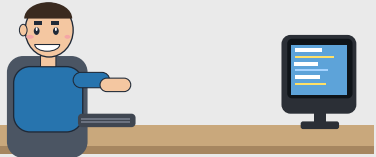
Source: H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu. 2026. Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. MSR '26

Be Aware of the Illusion of AI Productivity

- AI initially gives a solution that looks “solid” which makes us feel very productive

Days before OpenAI

Developer coding
2 hours

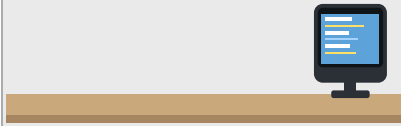


Developer debugging
6 hours

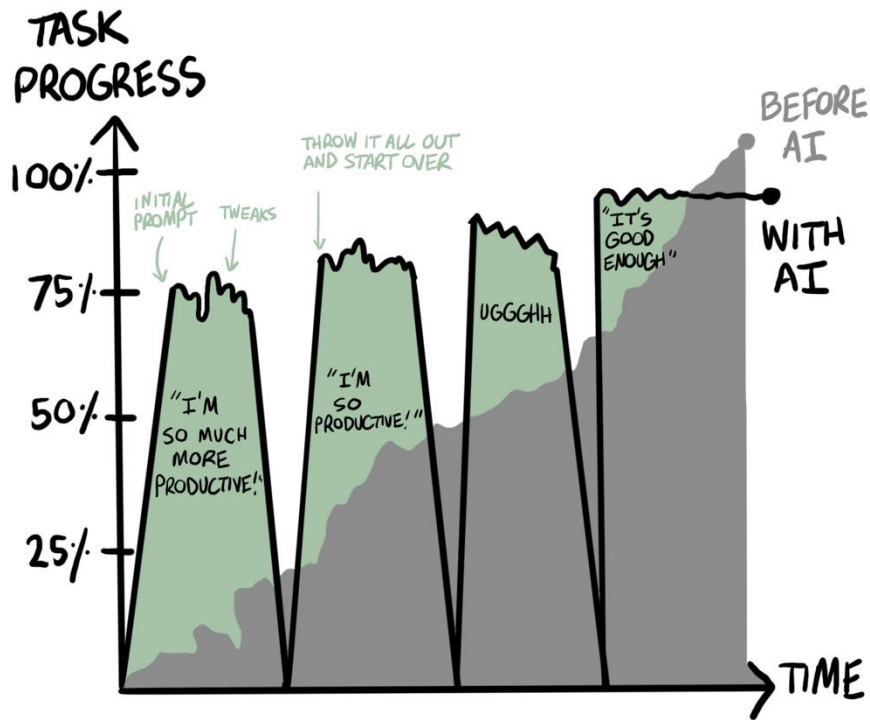


Days after OpenAI

ChatGPT generates code
5 minutes



Developer debugging
24 hours



THE ILLUSION OF AI PRODUCTIVITY

@forrestbrazeal

How AI Impacts Skill Formation

Think-Pair-Share: How do these results impact your approach to working with GenAI?
Discuss actionable insights!

- **Methods:** Controlled experiment let 52 developers code with a new library with/without AI and tests skill formation via questions and debugging tasks
- **Results:** Participants who relied on AI performed much worse than participants who did not use AI when debugging or explaining how to use the library
 - **Anti-Pattern:** Using AI for “**cognitive offloading**” (wholly or mostly relying on AI to solve the task) **minimizes learning effects**
 - **Better: Conceptual Inquiry** (treating AI like a “TA”) – ask only conceptual questions and reply on improved understanding to complete the task. Although this group encountered many errors, they also independently resolved these errors. On average, this mode was the fastest among high-scoring patterns and **second fastest** overall after the AI Delegation mode

Source: S. J. Hanwen, and A. Tamkin (Anthropic). "How AI impacts skill formation." *arXiv* (2026).

Key Takeaway

In the hands of an **expert**, AI is a very **powerful tool**
that creates efficiency

In the hands of a **novice**, AI creates the **illusion** of
competence that creates more **problems** than it solves

How to Use Generative AI The Right Way?



Generative AI is The Most Patient Pair Programmer

Now apply the **experience** you've gain with **Pair Programming** to AI!

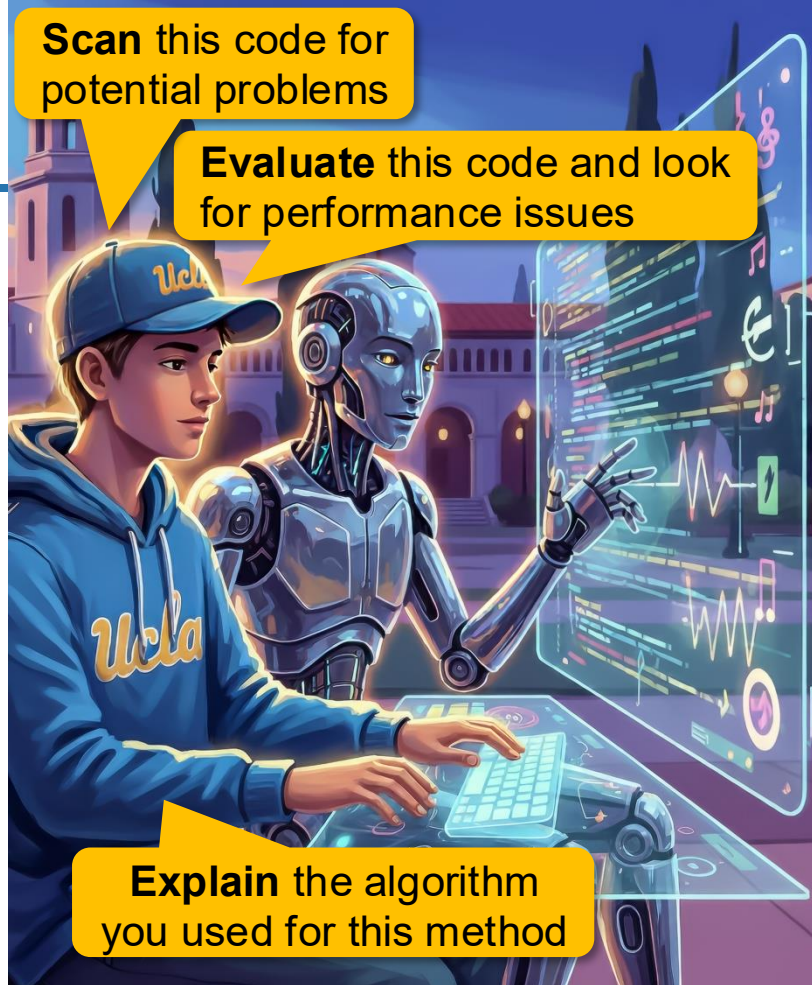
AI Pair Programming Mentality

As the **Driver** (You write the code):

- Ask AI to **critique** the code you wrote and ask for **suggestions** to improve it

As the **Navigator** (You let AI write the code):

- Make sure you **understand** all code
- Critically **review AI-generated code**
- Ask AI to **explain** their code



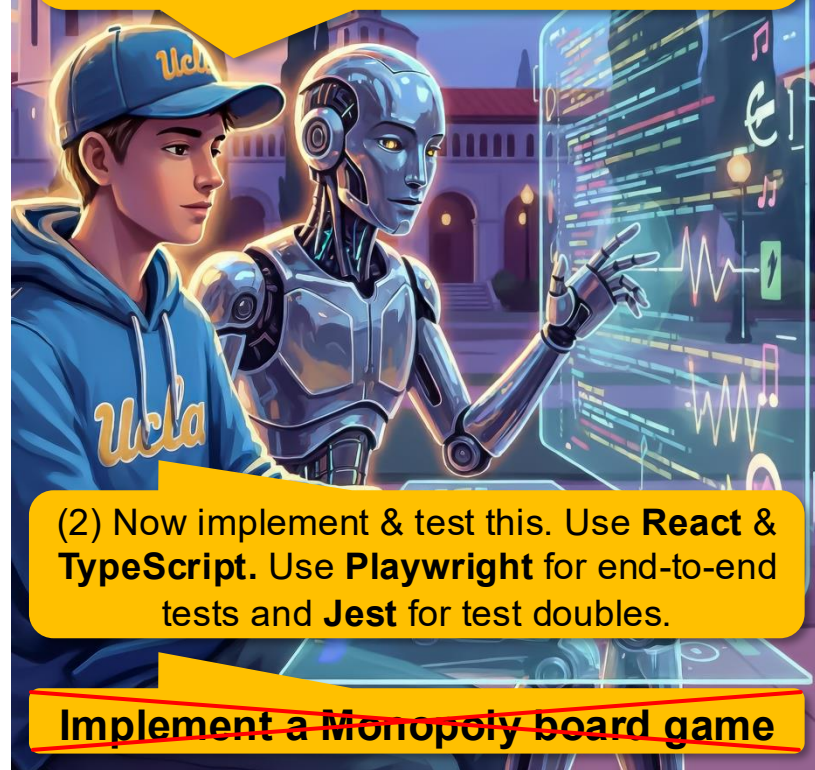
Generative AI is Your Knowledgeable Intern

AI is like a very knowledgeable intern who is **not good at applying their knowledge** to the right context. You should **orchestrate** their skills.

Iteratively Prompt AI with Concrete Steps

- Define **problem & high-level strategy**
- You pick the overall **architecture, design patterns**, how **components interact**, packages, and frameworks
- Before code generation, ask the AI to **explain its suggested approach** to critique & ask for specific refinements

(1) Implement Monopoly with the player using the **state pattern** (InJail, Bankrupt, NormalTurn). **Before you write code, first show me a class diagram.**



(2) Now implement & test this. Use **React & TypeScript**. Use **Playwright** for end-to-end tests and **Jest** for test doubles.

~~Implement a Monopoly board game~~

Find the Mistakes Copilot Made

```
class Player:
```

```
def roll_dice(self) -> int:
    die1 = random.randint(1, 6)
    die2 = random.randint(1, 6)
    return die1 + die2
```

```
class InJailState:
```

```
def take_turn(self, player: "Player") -> None:
    roll = player.roll_dice()
```

```
# Simulate checking for doubles
# (simplified: 50% chance)
```

```
import random
```

```
doubles = random.random() < 0.5
```

```
if doubles:
```

```
    print(f"{player.name} rolled doubles
          and is released from jail!")
```

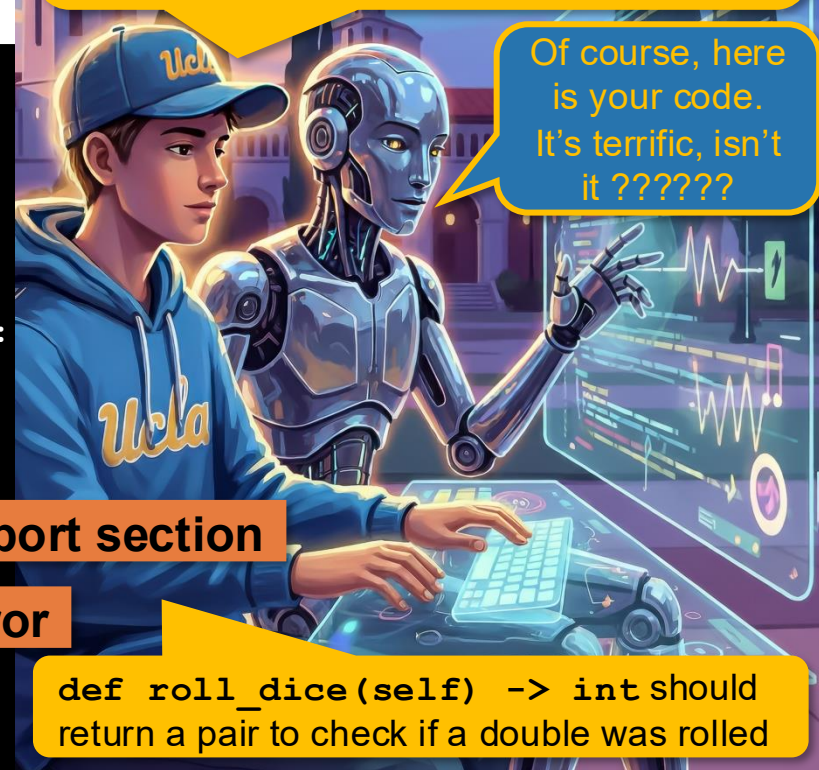
Should be in import section

Logical Error

Implement monopoly with the player using the **state pattern** (InJail, Bankrupt, NormalTurn). **Before you write code, first show me a class diagram.**

Of course, here is your code. It's terrific, isn't it ??????

def roll_dice(self) -> int should return a pair to check if a double was rolled



The AI Pair Programmer Successfully Fixed it!

```
class Player:
```

```
def roll_dice(self) -> tuple[int, int]:  
    die1 = random.randint(1, 6)  
    die2 = random.randint(1, 6)  
    return (die1, die2)
```

```
class InJailState:
```

```
def take_turn(self, player: "Player") -> None:  
    die1, die2 = player.roll_dice()  
    roll = die1, die2  
  
    print(f"{player.name} rolled a {die1} and  
          {die2} (total: {roll})")  
  
    if die1 == die2:  
        print(f"{player.name} rolled doubles  
              and is released from jail!")
```

Implement monopoly with the player using the **state pattern** (InJail, Bankrupt, NormalTurn). **Before you write code, first show me a class diagram.**

Oops, my bad.
Here is the
correct code

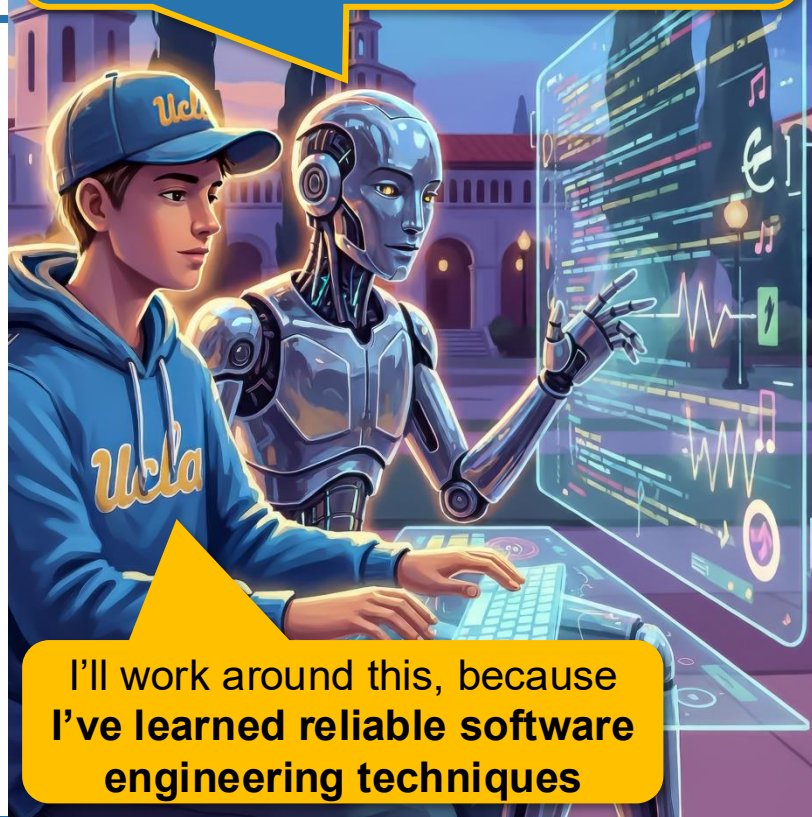
`def roll_dice(self) -> int` should
return a pair to check if a double was rolled

Adopt these Rules for Best Results

- **Mandatory Code Review:** Adopt a strict rule: "Always Review AI-Generated Code"
Every block of code an AI produces should be scrutinized as if an unreliable teammate wrote it
- **The Explainability Rule:** Never commit AI-generated code that you could not comfortably explain to a teammate

Source: Stephane H Maes. "The Gotchas of AI Coding and Vibe Coding. It's All About Support And Maintenance" April 2025

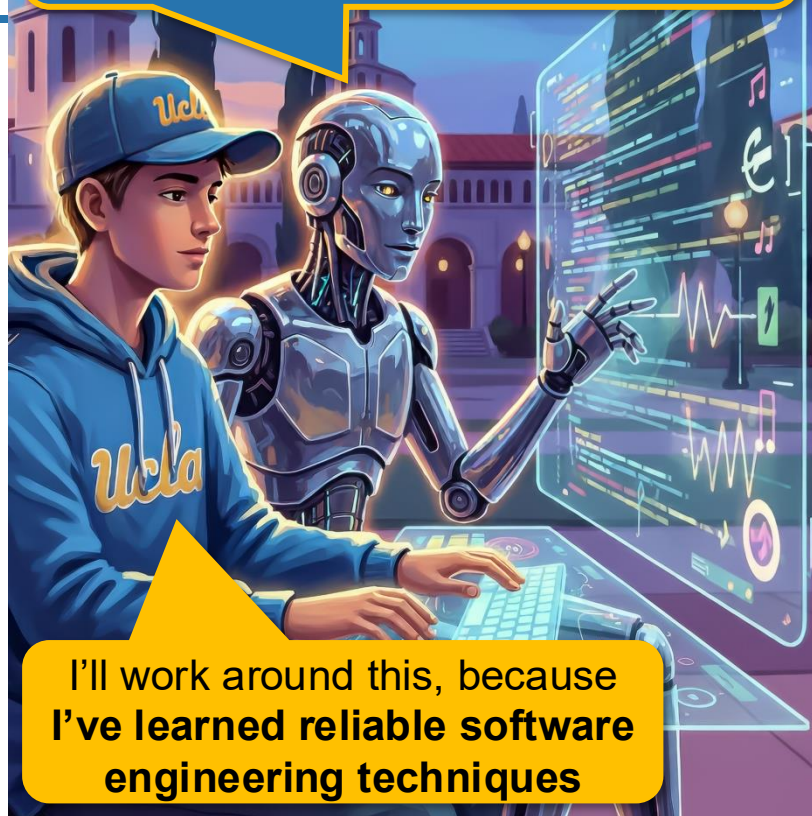
Assume everything I do is subtly incorrect, buggy, and insecure



Lesson Learned: Always double-check AI Code

- A lot of the code in the training set has many code smells.
 - **LLMs reproduce these code smells.**
- The LLM does not *understand* how the world works. It looks for linguistic similarities and correlations between the prompt and data in its training set
- **This is why verifying LLM-generated code is soooo important!**

Assume everything I do is subtly incorrect, buggy, and insecure



How can we Effectively Use Unreliable AI???

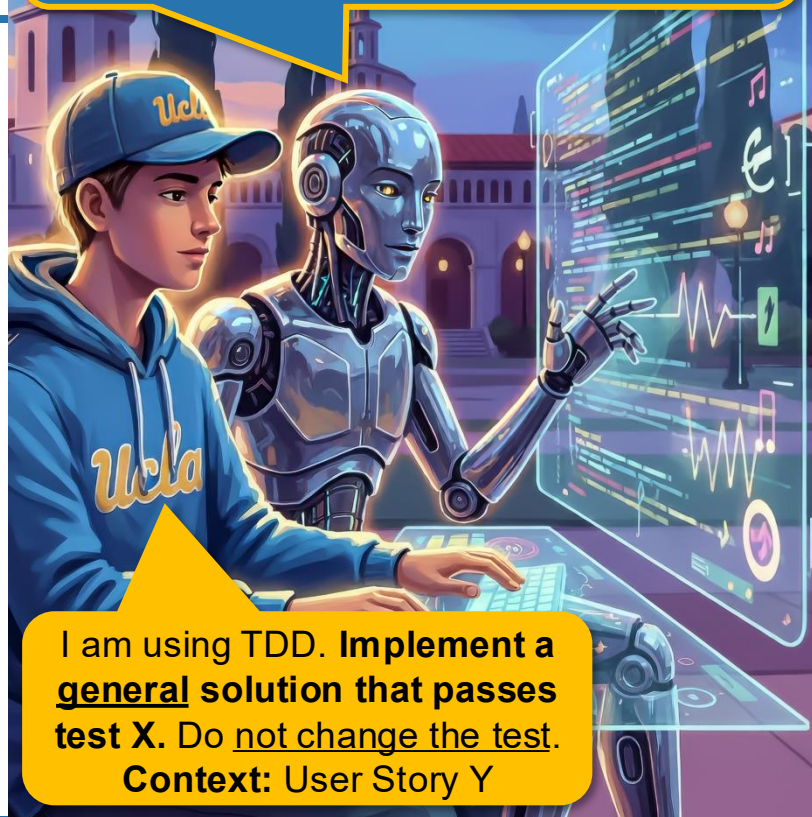
Make sure you can **verify the output** of genAI and that it **didn't break existing** code.

Test-Driven Development with GenAI

- Write a **test** (or more) just as if you were to implement the next feature yourself
 - AI may help you with test setup and infrastructure code
- Prompt the AI to make the test **pass**
- **Refactor AI-written code** (important)

See <https://github.blog/ai-and-ml/github-copilot/github-for-beginners-test-driven-development-tdd-with-github-copilot/>

Assume everything I do is subtly incorrect, buggy, and insecure



I am using TDD. Implement a general solution that passes test X. Do not change the test.
Context: User Story Y

TDD with LLM-based Code Generation

- Provide the LLM with (a) a problem statement, and (b) a test suite
- Merely **providing the LLM with tests** during generation **improves correctness**
- Adding a **remediation loop** that provided the LLM with the stack trace of failed tests **increases correctness** even more

See: [N. S. Mathews and M. Nagappan. Test-Driven Development and LLM-based Code Generation. ASE '24](#)

Assume everything I do is subtly incorrect, buggy, and insecure



Test-Driven Generation

- 1. Prompt the AI to **generate tests** for a given specification
- 2. Carefully **review the tests** to check if they are an adequate specification of the problem
- 3. Prompt AI to **generate the implementation** for the generated tests
- 4. Prompt the AI to **refactor** the code
- 5. **Review** the generated implementation to **check for overfitting**

See: <https://dzone.com/articles/test-driven-generation>

Assume everything I do is subtly incorrect, buggy, and insecure



YOU are the **supervisor** AI is your **junior developer**

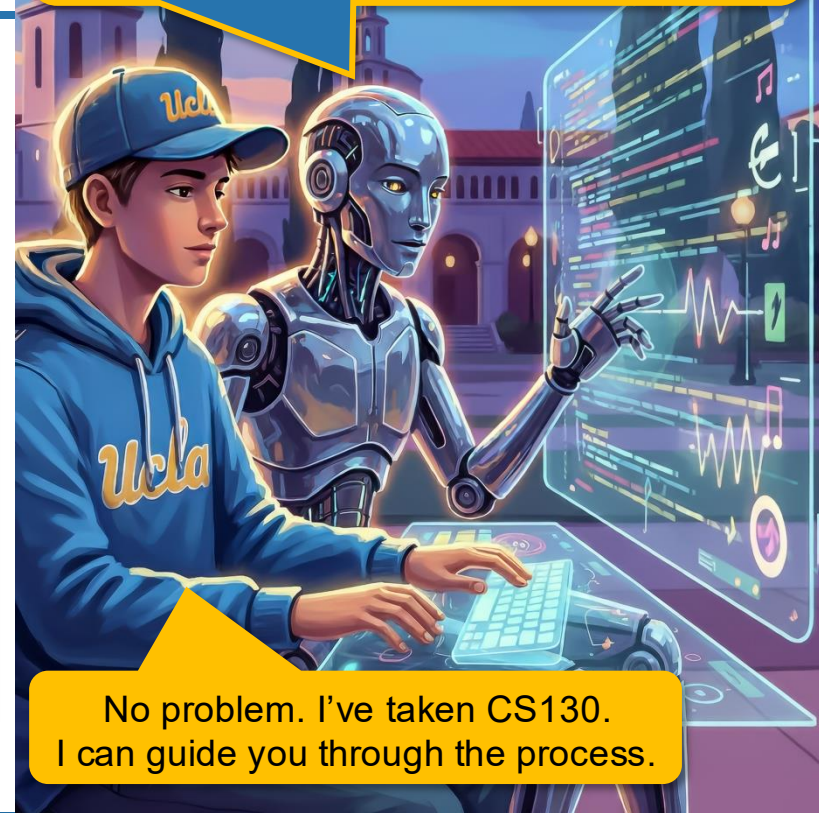
AI is like a very knowledgeable intern who is **not good at applying their knowledge** to the right context. You should **orchestrate** their skills.

Supervisor Mentality

You are **fully responsible** for all code (including bugs & security vulnerabilities)

- It is your job to **read & verify** the correctness of AI-generated code
- Always **test** all code written by AI
- Always **refactor** low-quality code
- Optimize important **quality attributes**

Sorry, I haven't taken CS130. So, I **don't know how to design & test high-quality software**. But I can write code very fast!



No problem. I've taken CS130. I can guide you through the process.

Which Coding Tasks are Well-suited for AI?

Where AI **shines**:

- Agents **Accelerate Straightforward, Repetitive, Scaffolding** Tasks
- Suitable for Following **Well-Defined** Plans
- Suitable for General SWE Tasks: Writing Tests, Refactoring, Documentation, Simple Debugging
- Suitable for Prototyping and Experimenting

Where it **stumbles**:

- Unsuitable for **Business Logic** and Tasks Requiring **Domain Knowledge**
- Unsuitable for **Complex, Stateful** Tasks
- Performance-Critical Deployment, **High-Stakes**, and **Security-Critical** Situations
- Anything requiring **real systems thinking** [and new architectures etc].

Ideal use case: Tasks that are **tiresome to solve** but **easy to verify** and when the **cost of an agent being wrong** is **low**.

Source: [R. Huang, A. Reyna, S. Lerner, H. Xia, B. Hempel. "Professional Software Developers Don't Vibe, They Control: AI Agent Use for Coding in 2025". arXiv preprint \(2025\)](#)

Prompt Engineering Patterns for Complex Prompts (Larger or Difficult Coding Tasks)

- **Introduction:** Give the AI an imaginary ‘role’ to think of themselves in.
E.g., “Role: Act as a diligent **expert** software engineer. You have **expertise** in Python and design patterns, clean code, and testing.”
 - Results in better responses, because the AI correlates words in the prompt with words mentioned in the training set. This way, it “**activates**” **specific knowledge and skills**
- **Concrete Task:** Tell the AI **what** and **where** it should implement code.
E.g., “Task: Implement the <feature> in <file>”
- **Contextual Information:** Provide the AI with enough information about what system this feature is used in, or what are important requirements for the feature.
E.g., “Context: This feature is part of this **user story** <user story>. We value **security & privacy over performance**.”
- **Instructions:** Explicitly list all steps. E.g., “Instructions: (1) plan a clean design, then (2) generate the code to write the program, and then (3) write unit tests & end-to-end tests.”

Prompt Engineering Tips

- Keep your AI prompt task **Simple**
 - The larger the amount of code that genAI should produce in one go, the more likely it is that the result is incorrect or low-quality
 - AI has **limited context size & limited reasoning capabilities**
 - Break up a task into **smaller & simpler tasks** if possible
 - “Please only describe the design & your reasoning, don’t implement code yet”
 - “Now please implement only <feature 1>, nothing else yet”
- Be **Specific**
 - Tell the AI all important **success criteria**
 - Fortunately, you’ve learned how to write good **Acceptance Criteria** already
- **Provide the exact context needed to solve the task (nothing more, nothing less)**
 - Addresses **context degradation**

Design Decision Prompts

- Besides Coding, AI can help you make design decisions. You can ask it to **describe the pros and cons of different design alternatives**
 - Tip: Ask **general question** about consequences of design rather than whether the decision would be ideal for your concrete scenario:
 - **Don't**: “Should I use the State design pattern **for this class**?”
 - LLMs currently do not *understand* design patterns and do not *understand* your context
 - **Better**: “What are **common consequences** for using the State design pattern?”
 - LLMs are great at reproducing general pros and cons.
- Then **it's your job to make the decision**

Question-Prompts

- You can tell the AI to ask you **relevant questions** that then help the AI find the right solutions
 - E.g., “[...] Before you start your task, **ask me any questions you have about important decisions, or requirements**”
 - E.g., “[...] Before you start your task, **ask me ordered Yes / No questions you needed to solve the task**”

“Replace the TODO” Prompt Pattern

- Key idea: give the AI a code skeleton that allows you to keep the control over the design
- Add todo notes with instructions right in the code for the AI to replace

```
class Player:
```

```
def roll_dice(self) -> tuple[int, int]:  
    die1 = #TODO fill in random die roll  
    die2 = #TODO fill in random die roll  
    return (die1, die2)
```

```
class InJailState:
```

```
def take_turn(self, player: "Player") -> None:  
    #TODO fill in the in-jail behavior
```



Resolve all remaining todos by filling in the right code. Ask me before making changes to other code

Essential Skills for Coding with AI

- **Delegation and agent orchestration**
 - Break down tasks, precisely articulate criteria, constraints, and context
- **Verification and quality assurance**
 - **review**, **test**, and **verify** AI-generated code
- **Product understanding, requirements engineering, and specification**
 - apply systems thinking (looking at the product as a whole)
- **Architecture and systems design**
 - overall system architecture, design patterns, how different components interact

Testing

User Stories

Up next...

“The future belongs to developers who can **model systems**, **anticipate edge cases**, and **translate ambiguity into structure**—skills that AI can’t automate”

Source: <https://ashtom.github.io/developers-reinvented>

Please fill out your Exit Tickets on Bruin Learn!

Question 1

1 pts

In your own words, please summarize **three key insights** you learned about **Software Engineering with Generative AI** today. (Do not copy phrases from the lecture slides)

Question 2

1 pts

For a particular coding task that you are currently working on for your course project,
(1) Write **an AI prompt that follows the best practices of this lecture** that helps you solve this task. (2) **Explain which best practices** this prompt follows.

Question 3

Please leave any questions that you have about today's material and things that are still unclear or confusing to you (if none, simply write N/A)

Credits: These slides use images generated with Gemini

Recommended Resources

- **“Developers, Reinvented”** by Thomas Dohmke
 - <https://ashtom.github.io/developers-reinvented>
- **“GitHub CEO predicts the future of programming...(Full Interview)”**
 - <https://www.youtube.com/watch?v=3WNukz5-Ch0>
- **“Good Vibrations? A Qualitative Study of Co-Creation, Communication, Flow, and Trust in Vibe Coding”** by Veronica Pimenova et al.
 - <https://arxiv.org/abs/2509.12491>